Computer vision - Course Notes (Week 6)

Michael Burke (based on slides from Tom Drummond and Maxine Tan)

January 2022

1 Week 6: Multi-view geometry and 3D reconstruction

So far, we have mostly been concerned with single view operations (or in the case of homographies multi-view operations where points lie on planes). We now turn to the general case, two images of the same scene, captured at different viewpoints. Doing so allows us to track 3D information of the scene (for example in motion capture systems), or to say something about how a camera has moved from one frame to another.

Let's start by looking at the most general case.

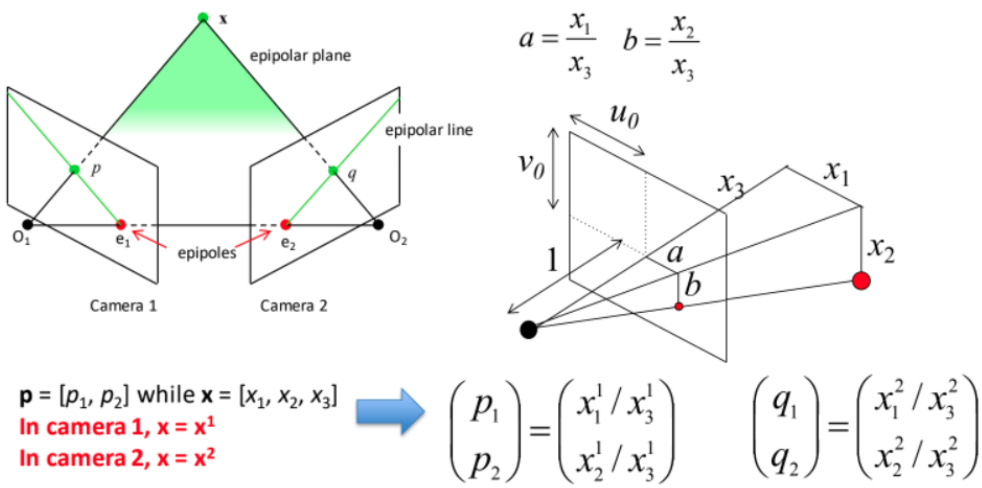

Figure 1: Multiple view geometry

Assume we have some point $\mathbf{x} = [x_1, x_2, x_3]^{\mathrm{T}}$ in 3D space, that is viewed in camera 1, with optical point $\mathbf{O}_1$. We'll assume our camera has normalised coordinated (focal length = 1), this means the point $\mathbf{x}$ projects to point $\mathbf{p} = [x_1/x_3, x_2/x_3]^{\mathrm{T}}$ in the first image. and that point $\mathbf{x}$ is measured in the coordinate frame of camera 1.

Now let's assume we have another camera, with optical point $\mathbf{O}_2$, that views the same point $\mathbf{x}$. In camera 2, the point projects to coordinates $\mathbf{q}$. Let's also assume that the transformation between camera 1 and 2 is $\begin{bmatrix}\mathbf{R} & \mathbf{t}\end{bmatrix}^{\mathrm{T}}$.

This means that the coordinates of point $\mathbf{x}$ in camera 2 are:

$$\mathbf{x}^2 = \mathbf{R}\mathbf{x} + \mathbf{t} \tag{1}$$

Similarly for point $\mathbf{p}$

$$\mu\mathbf{q} = \lambda\mathbf{R}\mathbf{p} + \mathbf{t} \tag{2}$$

where λ and μ are some unknown scales. So $\mathbf{q}$ is a linear combination of vector $\mathbf{R}\mathbf{p}$ and vector $\mathbf{t}$.

Looking at the figure above, we can imagine a plane passing through the image points $\mathbf{p}, \mathbf{q}$ and the optical points and connecting to the 3D point $\mathbf{x}$. We call this the *epipolar plane*. If we imagine that we detecting some point of interest at $\mathbf{p}$. but didn't know how far away it was, this picture tells us that we can find the same point along a line in our second view - the *epipolar line*. This is an interesting constraint, a point in one view will have a correspondence in another view on the epipolar line.

If we had a known correspondence between $\mathbf{p}$ and $\mathbf{q}$, we can triangulate the distance to point $\mathbf{x}$. We can use this constraint to make it easier to find correspondences.

We can also draw a line between the optical centres of the cameras, vector $\mathbf{t}$. This line passes through the *epipoles*, points of intersection with the image plane in each view. The epipoles lie on the epipolar lines, along with the point $\mathbf{q}$.

Recall that the equation of a line through two points is given by the cross product, so we can form the equation of the epipolar line as

$$l = \mathbf{t} \times \mathbf{R}p \tag{3}$$

because we don't care about the scale ($\mathbf{t} \equiv \mathbf{e}_2$). From our vanishing point notes this line is normal to the plane formed by the epipolar line and the optic centre of the image.

We can re-write the cross product as a matrix multiplication

$$\mathbf{t} \times \mathbf{x} = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{4}$$

, which gives us a new form of the epipolar line equation

$$\mathbf{l} = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix} \mathbf{Rp} \tag{5}$$

We call the matrix

$$\mathbf{E} = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix} \mathbf{R} \tag{6}$$

the essential matrix.

The essential matrix transforms a point in camera 1 to a line in camera 2.

$$\begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix} = \mathbf{E} \begin{bmatrix} p_1 \\ p_2 \\ 1 \end{bmatrix} \tag{7}$$

In camera 2, point $\mathbf{q}$ must lie on this line.

$$\begin{bmatrix} q_1 \\ q_2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix} = 0 \tag{8}$$

This produces the epipolar constraint:

$$\begin{bmatrix} q_1 & q_2 & 1 \end{bmatrix} \mathbf{E} \begin{bmatrix} p_1 \\ p_2 \\ 1 \end{bmatrix} = 0 \tag{9}$$

or

$$\begin{bmatrix} p_1 & p_2 & 1 \end{bmatrix} \mathbf{E}^{\mathrm{T}} \begin{bmatrix} q_1 \\ q_2 \\ 1 \end{bmatrix} = 0. \tag{10}$$

if $\mathbf{p} = e_1$, the cross product between $\mathbf{t}$ and $\mathbf{Rp}$ is zero. This means the epipole e_1 is in the null space of $\mathbf{E}$.

The essential matrix has 5 degrees of freedom (3 in the rotation, and 3 in the translation, but the scale of $\mathbf{E}$ doesn't matter). This means we can compute E from 5 point correspondences.

1.1 The Fundamental matrix

So far we have assumed normalised coordinates. We can rewrite the epipolar constraint to include camera calibration matrices

$$\begin{bmatrix} q_1 & q_2 & 1 \end{bmatrix} \mathbf{K}_2^{-\mathrm{T}} \mathbf{E} \mathbf{K}_1^{-1} \begin{bmatrix} p_1 \\ p_2 \\ 1 \end{bmatrix} = 0 \tag{11}$$

$$\begin{bmatrix} q_1 & q_2 & 1 \end{bmatrix} \mathbf{F} \begin{bmatrix} p_1 \\ p_2 \\ 1 \end{bmatrix} = 0 \tag{12}$$

where $\mathbf{F} = \mathbf{K}_2^{-\mathrm{T}} \mathbf{E} \mathbf{K}_1^{-1}$ is the *fundamental matrix*[1]. For any two pictures of the same scene, there is a 3x3 fundamental matrix that relates pixel coordinates in the two images.

1.2 The 8-point algorithm

The fundamental matrix has 7 degrees of freedom, although it has 9 values, we save 1 degree of freedom because of scale, and another because this matrix as an eigenvalue equal to zero. We use an 8-point algorithm to find F though, as this is a simple method.

Writing out the epipolar constraint for each correspondence

$$\begin{bmatrix} q_1 & q_2 & 1 \end{bmatrix} \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ 1 \end{bmatrix} = 0 \tag{13}$$

gives the equation

$$p_1 q_1 F_{11} + p_2 q_1 F_{12} + q_1 F_{13} + p_1 q_2 F_{21} + p_2 q_2 F_{22} + q_2 F_{23} + p_1 F_{31} + p_2 F_{32} + F_3 3 = 0 \tag{14}$$

Re-writing this as a matrix multiplication gives

$$\begin{bmatrix} p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{bmatrix} = 0 \tag{15}$$

So with 8 correspondences we get the matrix:

$$\begin{bmatrix} p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \\ p_1 q_1 & p_2 q_1 & q_1 & p_1 q_2 & p_2 q_2 & q_2 & p_1 & p_2 & 1 \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{bmatrix} = 0 \tag{16}$$

with each row a different correspondence p and q. As before, $\mathbf{F}$ lies in the null space of this matrix.

We can decompose the Fundamental matrix to find the rotation and translation between two views. We can also use the fundamental matrix to triangulate (compute the 3D location of these correspondences. This is often called *structure and motion* and the basis of a number of simultaneous localisation and mapping schemes.

[1] `https://www.youtube.com/watch?v=DgGV3l82NTk`

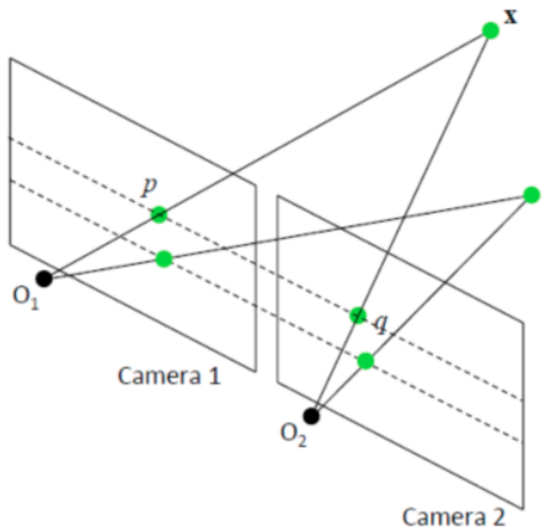

Figure 2: Epipolar lines are horizontal for parallel cameras.

1.3 Stereo Reconstruction

So far, we have considered the general case of multi-view 3D reconstruction. Things are easier if we consider a stereo camera pair of parallel cameras. Here, the rotation between views is **I** and the translation is only on the x-axis, so $\mathbf{t} = [a, 0, 0]^{\mathrm{T}}$. So for parallel cameras the essential matrix is

$$\mathbf{E} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -a \\ 0 & a & 0 \end{bmatrix} \equiv \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \tag{17}$$

This means that the epipolar line is horizontal $\mathbf{l} = \mathbf{E}\mathbf{p} = [0, -1, v_1]^{\mathrm{T}}$. So a point in one image will lie in the same row in the second image.

This also means that all the information required to tringulate depth is in the u coordinates. We use similar triangles to do this. Looking at the figure below, we can see that $\frac{u_1}{1} = \frac{a_1}{z}$ and $\frac{u_2}{1} = \frac{a_2}{z}$. This means that $\frac{1}{z} = \frac{u_1 - u_2}{a}$. We call $u_1 - u_2$ the *disparity*. 3D reconstruction is easy in this case, for each epipolar line pair, find correspondences, compute the disparity, and invert this.

Unfortunately, finding correspondences can be tricky. We could use some of the matching approaches from earlier in the unit, but that is computationally expensive. Another option is to make some assumptions about the object being imaged (eg. it is convex).

The accuracy of our reconstruction is related to the baseline - we get more accurate depth estimation with a wider baseline. However, this makes finding correspondences harder, so we have an engineering tradeoff when designing stereo camera pairs.

1.4 Bundle adjustment

So far, we have considered 3D reconstruction using pairs of cameras/ views. But sometimes, we have multiple images captured at multiple views. Bundle adjustment refines a visual reconstruction to produce jointly optimal 3D structure and viewing parameters. The term bundle refers to the light rays leaving each 3D point.

The general approach in bundle adjustment is as follows:

- Calculate approximate camera geometry (eg. pairwise epipolar geometry)

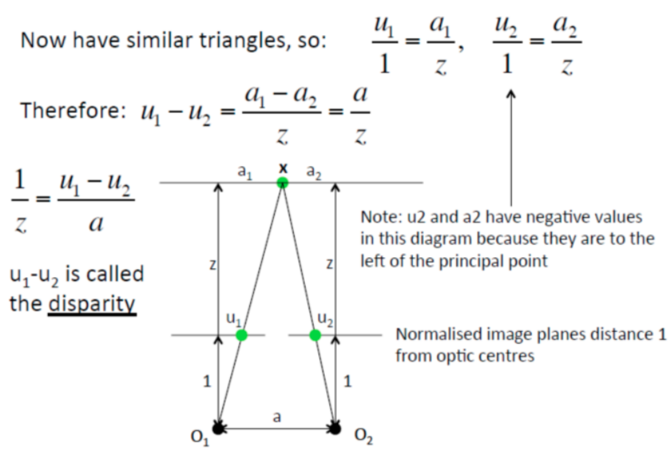

Figure 3: Stereo reconstruction using similar triangles.

- Calculate the approximate positions of all matched points by triangulation
- Calculate the re-projection errors for each point in each view
- Minimise the total reprojection error with respect to all these points and cameras using a suitable optimisation algorithm.
- Update the estimates of the camera poses and point positions.

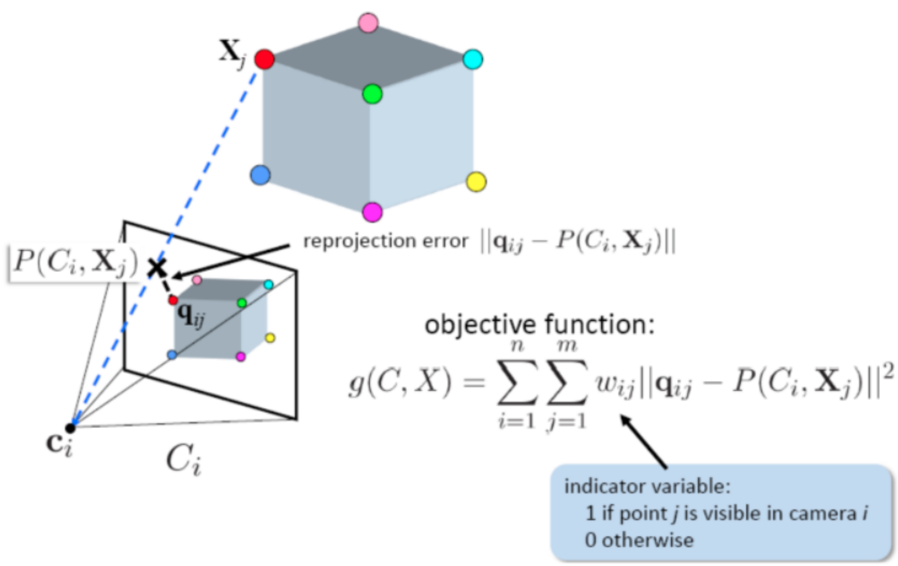

Figure 4: Bundle adjustment.

1.5 Space carving

So far we have considered 3D reconstruction using point-correspondences, but another approach is also effective, particularly when it comes to building 3D models of objects (eg. scanning objects for 3D printing.) We call this approach space or voxel carving.

Here, we start with a volume made of 3D voxels covering the area we have imaged. We then project each of those voxels into the various views, and check for consistency among the views. If the voxel is not visible in the scene, we remove it. If the projected voxel intensities disagree (not a correspondence) we remove the voxel. We continue to do so until we have carved our object our of the voxel grid.

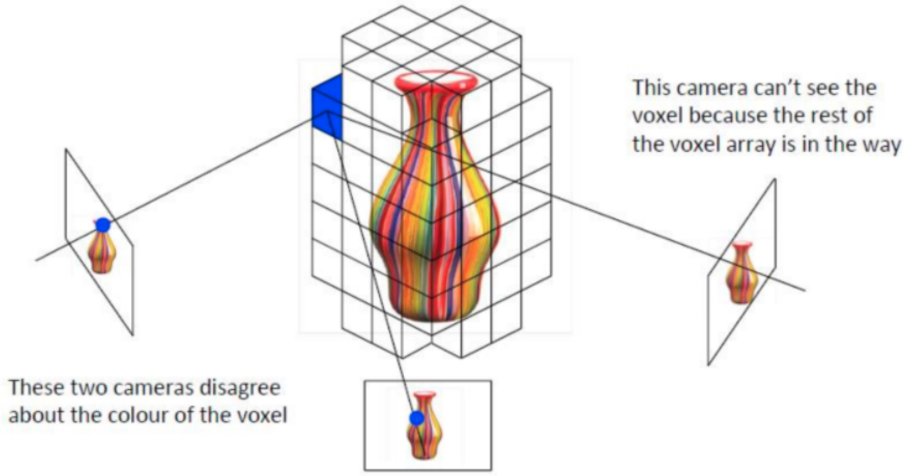

Figure 5: Space carving.

In robotics, this approach is sometimes called occupancy grid mapping. There are some issues with this approach (eg. it can' see inside objects), but it works well for statues and dinosaur models[2].

2 Summary

The first 6 weeks of this unit took us through the basics (images as 2D grids), models of light and radiance, operations (thresholding and filtering), representations (keypoints and masks), the factors of variation we need to cope with in computer vision. We looked a invariant matching strategies, and searching methods to speed these up (decision trees, kd-trees). We also looked at the geometry of computer vision, how points on planes can be mapped between images (useful for remote sensing, augumented reality, image stitching, and visual odometry or place recognition for aerial vehicles), and constraints that arise from this geometry. We saw how we could exploit these constraints for model-based computer vision using ransac, or for 3D reconstruction from multiple views.

2.1 Exam expectations (Weeks 1-6)

For your exam:

- We expect you to be able to explain how you may solve new computer vision problems using the techniques you have been taught (pseudo code level descriptions, listing steps)
- We will expect you to perform certain mathematical operations by hand (within reason), to demonstrate your understanding of these concepts (eg. light models, convolutions, matrix multiplication for projection, homographies, various decompositions, lines and multi-view geometry constraint tests, vector operations like dot and cross products.
- We will expect you to answer theoretical questions about these algorithms and techniques.

[2]`https://blogs.mathworks.com/loren/2009/12/16/carving-a-dinosaur/`

2.2 What's next?

Over the final 6 weeks, you will be going through approaches focused on interpreting the contents of images, specifically looking at unsupervised learning or clustering for segmentation/ image generation for images (and different representations), supervised learning for object detection and image classification.